Metanet Developer

Developer Integration Guide

Welcome, developer! This guide provides everything you need to integrate your application with the Metanet.page platform. Integration allows your app to interact with the user's wallet, create posts, and much more.
Apps on the Metanet.page platform run in a secure iframe environment, ensuring that your app can interact with the user's wallet without the user exposing sensitive information. This guide will walk you through the core messaging protocol, how to use our JavaScript SDK, and best practices for building robust applications. The sandboxed iframe only gives allow-script and allow itsself as origin permissions, ensuring a secure environment. No alerts, confirmations, or prompts are used, as all interactions are handled through the messaging protocol.
The core of our integration is a simple and robust messaging protocol based on window.postMessage . This makes it language-agnostic, allowing you to build apps in vanilla JavaScript, React, Vue, or any other framework that can run in a browser. For convenience, we also provide a lightweight JavaScript SDK to simplify these interactions.

The Core Protocol

Communication between your app (loaded in an iframe) and the Metanet.page host platform happens via messages. You don't need any special libraries, just the standard browser APIs.

Sending Messages to the Platform

To send a message, you use window.parent.postMessage. Every message must be an object with a specific structure:

```
window. parent.postMessage({
  command: "ninja-app-command", // This is always the command
  detail: {
        type: "connection", // The type defines the action
        // ...other properties for this type
  }
}, "*");
```

- `command:` Must always be the string "ninja-app-command". This identifies the message as intended for our platform.
- `detail:` An object containing the actual instructions.
  - `type:` A string that specifies the action you want to perform (e.g., "`connection`", "`pay`").
  - Additional properties depend on the `type` and are detailed in the Command Reference.

Receiving Messages from the Platform

The app will load inside the iframe appending all query parameters from the URL to the `window.location.search.` The platform automatically adds a `metanetLng` parameter with the user's language preference (e.g., ? `metanetLng=en`). If the user is anonymous, the platform will send a `connection-response` message with an `anonymous: true` property and no wallet property. If the app is being loaded from a Premium Username, the connection-response payload will also include `appPageSchema` containing the Username Schema object that can be found in any Premium username's Verified Card by clicking the Verified blue badge. This schema can be used by the app to identify where it was loaded from and access profile information.

Apart the query on app load, the platform will respond to the app's commands as descrived further below.

To receive responses, you listen for `message` events on the `window` object. It's crucial to validate that the message is from the platform by checking its structure.

```
window.addEventListener("message", (event) => {
  // IMPORTANT: Add origin check in production for security
  // if (!
["https://www.metanet.page","https://www.metanet.ninja"].includes(event.orig
in)) return;

  const { command, type, payload } = event.data;

  if (command === "ninja-app-command") {
    console.log("Received message from platform:", { type, payload });

    if (type === "connection-response") {
      // Handle the connection response
    } else if (type === "pay-response") {
      // Handle the payment response
    }
    // ... and so on for other response types
  }
});
```

JavaScript SDK (Convenience Wrapper)

To make development faster, you can use our lightweight JavaScript SDK. It's a simple wrapper around the Core Protocol that provides two main functions: sendCommand and onCommand. It handles the postMessage structure and

event listening for you.
This is not a package to install. Simply include this script in your app's HTML or main JavaScript file.

```
<script type="module">
  // Create a shared event target to manage commands within your app
  const eventTarget = new EventTarget();

  // Wraps window.parent.postMessage with the required structure
  const sendCommand = (commandObj) => {
    try {
      window.parent.postMessage({
        command: "ninja-app-command",
        detail: commandObj // commandObj is { type: "...", ... }
      }, "*");
      // Also dispatch locally for your app's own listeners
      eventTarget.dispatchEvent(new CustomEvent('platform-command', {
detail: commandObj }));
    } catch (err) {
      console.error('Error sending command:', err);
      throw err;
    }
  };

  // A listener that centralizes responses from the platform
  const onCommand = (callback) => {
    window.addEventListener('message', (event) => {
      // Optional: Check event.origin for security
      if (event.data && event.data.command === "ninja-app-command") {
        callback(event.data); // callback receives { command, type, payload
}
      }
    });
  };

  // Expose the SDK to your app
  window.platformSDK = { sendCommand, onCommand };
</script>
```

With this script included, you can access the functions via `window.platformSDK.sendCommand(...)` and `window.platformSDK.onCommand(...)`.

Typical Integration Flow

A standard interaction between your app and the platform follows these steps:

1. **Establishing Connection:** On launch, your app sends a message with type: "connection". The platform replies with a "connection-response" message containing the user's wallet details and other essential data. This data should be stored and used to verify subsequent messages.
2. **Requesting an Action (e.g., Payment):** To initiate a payment, your app sends a message with type: "pay" and a payload detailing the recipients and amounts.
3. **Handling the Response:** The platform processes the request (which may require user confirmation) and replies with a "pay-response" message. Your app must check this response to confirm if the payment was successful.
4. **Submitting Content:** Suppose you wanted to change the user for the tools you provided and now you are ready to let them post the produced content on their Feed. After any required payment authorisations are confirmed, you can broadcast or keep for later, then your app can send a message with type: "create-post" to publish content. The platform will confirm the result with a "create-post-response".

Command Reference

This section details the `type` and `detail` object for each message you can send, and the `payload` for each response you can receive.

Connection & Authentication

type: "connection" (App → Platform)
Send on app launch to establish the connection. The optional `navbg` field can suggest a background color for the platform's navigation bar.

```
// Message detail object
{
  type: "connection",
  navbg?: string // Optional: preferred navigation background color
}
```

type: "connection-response" (Platform → App)
The platform's response to a connection request. Contains credentials for validating future messages. We show the full structure here, but you only need to use the payload or in this case exceptionally also the `icIdentityPackage` - for all other types you only need the response's payload.

```
// Response full object
{
  command: "ninja-app-command",
  type: "connection-response",
  payload:{
    appId: string,
    timestamp: number, // Unix timestamp in milliseconds
```

```
    anonymous: true, // this is present and set to true when the user is not
logged in
    wallet: {
      address: string, // this is dereived from the publicKeyHex
      publicKeyHex: string, // Hex-encoded public key, this can be verified
against response signature
      rootPrincipal: string, // User's root ICP principal
    }
    icDelegation: JSON,   // Public delegation included in the signed
payload (time-bounded)
  }
  appPageSchema: {  // Present when loaded from a Premium Page
    "protocolId": "NINJA-NS",
    "symbol": "authenticator",
    "...": "...",       // Other schema fields
    "properties": {
      "username": "Authenticator", // Important: The Premium Username
      "profile": {
        "bio": "This is the official Authenticator app...",
        "...": "..."    // Other profile fields
      },
      "pubkey":
"0390247c087a3ae15504d2beadf6d8f137ab970ca3215197ce30864f26f7ab93ab",
      "signature":
"304402201de7751496743f56a4c4713117f532301f6a4c9b2cc635225e7145118ba07c1a022
03469738f3299ec0b78aeaa3438156336f7aa3f7d0b28a3a7d51d3ce487038d85",
      "...": "..."      // Other properties
    }
  },
  genericUseSeed: string, // Fixed 32-byte hex seed for app-specific
cryptographic operations
  icIdentityPackage: {   // Full delegation package including private key
    delegation: JSON,
    privateKey: string
  },
  signature: "dummy_signature" // Signature of the payload only
}
```

> **Key Field Explanations**
>
> **icDelegation:** A time-bounded delegation for ICP operations. This delegation expires periodically for security and should not be stored long-term. Use it for immediate ICP blockchain operations.

> **rootPrincipal:** The user's stable ICP principal identifier. Do not trust this piece of info. This only tells the logged in user claims to own that principal. But there is no guarantee this is true in this version of the SDK. Use it as certification of the user approving this Principal, potentially to receive ICP tokens in it, but not as proof of ownership of that address.
>
> **genericUseSeed:** A fixed 32-byte hex-encoded seed derived specifically for your app. This seed is deterministically generated from the user's identity and your app ID, providing a consistent cryptographic foundation for app-specific operations like key derivation, encryption, or signing. Unlike the time-bounded icDelegation, this seed remains constant for the user-app pair and can be safely stored for persistent cryptographic operations.
>
> **appPageSchema:** Only present when the app is loaded from a Premium Username page. Contains the complete Username Schema object including properties.username (the Premium Username) and properties.signature. For verification, validate the signature against all properties except the signature itself using properties.pubkey. This allows your app to authenticate which Premium Username it was loaded from.

Payments (BSV, ICP & KDA)

**type: "pay" **(App → Platform)
Requests a payment. Supports BSV, ICP token, and KDA payments. Each item in `recipients` can be a reason-only object for platform fees, or a full payment object.

BSV Payments (Default)

```
// BSV Payment (existing behavior)
{
  type: "pay",
  ref: "unique_reference", // A unique ID you create to track this request
  recipients: [
    { reason: "AI_IMG" }, // Platform fee for a specific service
    {
      address: "target_address", // Bitcoin address
      value: 330000, // in satoshis OR
      fiatValue: 12, // in fiat currency (platform will convert)
      currency: "USD", // optional, defaults to USD. Supports: USD, EUR,
GBP, JPY, AED, AUD, CAD, CHF, CNY, INR, BRL, MXN, KRW, RUB, ZAR, SEK, NOK,
DKK, SGD, HKD, NZD, THB, IDR, MYR, PHP, TWD, PLN, TRY, ARS, CLP, COP
      note: "optional_note"
    }
  ]
}
```

> **Important Currency Rules:**

> **Single currency only:** All recipients with fiatValue must use the same currency. Mixed currencies will produce an error.
> **Auto-conversion:** If any recipient specifies a currency, ALL recipients will display their fiat values in that currency to the user (even if they specified values in satoshis only).
> **Default:** If no currency is specified, USD is used by default.

ICP Token Payments (New)

For ICP token payments, include a token object with protocol specification. Currently supports single recipient only.

```
// ICP Token Payment
{
  type: "pay",
  ref: "unique_reference",
  token: {
    protocol: "ICP",
    specification: {
      ledgerId: "choose_from_supported_ledger_ids" // Canister ID of the
token ledger
    }
  },
  recipients: [
    {
      address: "target_principal", // Principal ID
      value: 1000000, // Amount in token's smallest unit (e.g., decimal with
up to 8 decimal digits for ICP)
      note: "Payment for premium service"
    }
  ]
}
```

Supported ICP Ledger IDs:

- `ryjl3-tyaaa-aaaaa-aaaba-cai` - ICP
- `xevnm-gaaaa-aaaar-qafnq-cai` - ckUSDC
- `6c7su-kiaaa-aaaar-qaira-cai` - GLDT
- `mxzaz-hqaaa-aaaar-qaada-cai` - ckBTC
- `ss2fx-dyaaa-aaaar-qacoq-cai` - ckETH

KDA (Kadena) Payments

For Kadena payments, set `protocol: "KDA"` in the `token` object. Currently supports single recipient only. The recipient address must be a Kadena `k:` account.

```
// KDA Payment
{
  type: "pay",
  ref: "unique_reference",
  token: {
    protocol: "KDA",
    specification: {
      chainId: "2" // Optional, defaults to "2". Kadena chain ID (0-19)
    }
  },
  recipients: [
    {
      address: "k:abc123...", // Kadena k: account
      value: 1.5, // Amount in KDA (decimal)
      note: "Payment for service"
    }
  ]
}
```

> **KDA Payment Notes:**
>
> **Amount format:** KDA amounts are in decimal (e.g., 1.5 = 1.5 KDA), unlike ICP which uses smallest units.
> **Chain ID:** Defaults to chain 2. Most KDA operations on the platform use chain 2.
> **Account format:** Recipient must be a valid Kadena k: account (prefixed with "k:").
> **Single recipient:** Only one recipient per payment request is supported.

type: "pay-response" (Platform → App)
Indicates the outcome of a payment request. Match it using the ref you provided. Response format varies depending on payment type (BSV, ICP, or KDA).

BSV Payment Response

```
// BSV Payment Success
{
  ref: "unique_reference",
  success: true,
  rawTxHex: "transaction_hex", // Bitcoin transaction hex
  responseCode: "OK_SUCCESS",
  timestamp: number, // Unix timestamp in milliseconds
}

// BSV Payment Failure
{
  ref: "unique_reference",
```

```
  success: false,
  rawTxHex: null,
  message: "Error message",
  responseCode: "ERR_ABORTED",
  timestamp: number, // Unix timestamp in milliseconds
}
```

@@@@@@@@@@@@@@@@@@@@@@@@@@@@

Broadcasting BSV Transactions

When you receive a successful BSV payment response, you get the rawTxHex - the signed transaction ready for broadcast. While you can broadcast this at your own pace, we strongly recommend using our overlay service for indexing and UTXO management. Please be aware we do NOT index transactions bigger than 33Kb.

Broadcasting API Call:

```
// Step 1: Create the payload
const payload = {
  data: {
    action: "broadcastTransactions",
    raws: [rawTxHex], // Array of transaction hex strings (single or
multiple)
    params: { source: "your-app-id", timestamp: Date.now() }
  }
};

// Step 2: Sign the payload for authentication
// You'll need elliptic and crypto-js libraries for signing
import { ec as EC } from 'elliptic';
import SHA256 from 'crypto-js/sha256';

const ecInstance = new EC('secp256k1');
const keyPair = ecInstance.keyFromPrivate(yourPrivateKeyHex);
const publicKeyHex = keyPair.getPublic(true, 'hex');

// Create canonical payload string (wrap data in { data: ... })
const canonicalPayloadStr = JSON.stringify({ data: payload.data });

// Hash and sign (CRITICAL: Buffer.from without 'hex' parameter!)
const hashHex = Buffer.from(SHA256(canonicalPayloadStr).toString());
const signature = keyPair.sign(hashHex, { canonical: true }).toDER('hex');

// Step 3: Make the API call
```

```
const response = await fetch('https://api.metanet.ninja/data/api', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'x-signature': signature,      // DER-encoded signature
    'x-pubkey': publicKeyHex       // Compressed public key (33 bytes, hex)
  },
  body: canonicalPayloadStr        // Send the canonical string (with data
wrapper)
});

const result = await response.json();

// Response format from Metanet API:
// {
//   success: true,
//   data: [
//     {
//       txid: "abc123...",           // Transaction ID
//       success: true,               // Broadcast status boolean
//       error: null                  // null if successful, error object if
failed
//     },
//     // ... one object per transaction in raws array
//   ]
// }
//
// Error response:
// {
//   success: false,
//   error: "Error message describing what went wrong"
// }
```

Response Details:
Success: Returns array of results, one per transaction
txid: The transaction ID if broadcast was successful
status: "success" for successful broadcasts
error: null on success, error object with details on failure
Authentication Requirements:
x-signature: DER-encoded ECDSA signature of SHA256(canonicalPayloadStr)
x-pubkey: Compressed public key (33 bytes hex) matching the signature
canonicalPayloadStr: JSON.stringify({ data: payload.data }) - wraps data object
Body: Send canonicalPayloadStr (same as what you signed)
Hash: Buffer.from(SHA256(canonicalPayloadStr).toString()) - NO 'hex' parameter!

Benefits: Our overlay service automatically indexes UTXOs, tracks transaction states, and provides better error handling. It uses the BSV P2P network with fallback mechanisms for reliable broadcasting.

ICP Token Payment Response

```
// ICP Payment Success
{
  ref: "unique_reference",
  success: true,
  transferOutcome: 12345, // ICRC-1 transaction index
  responseCode: "OK_SUCCESS",
  timestamp: number, // Unix timestamp in milliseconds
}

// ICP Payment Failure
{
  ref: "unique_reference",
  success: false,
  message: "Insufficient funds", // Error description
  responseCode: "ERR_ICP_FAILED", // or other error codes
  timestamp: number, // Unix timestamp in milliseconds
}
```

ICP-Specific Error Codes:

ERR_UNSUPPORTED_TOKEN - Ledger ID not supported
ERR_MULTIPLE_RECIPIENTS - ICP payments support single recipient only
ERR_MISSING_PARAMS - Missing recipient address or amount
ERR_ICP_PREP_FAILED - Failed to prepare ICP payment
ERR_ICP_FAILED - ICP transaction failed
KDA Payment Response

```
// KDA Payment Success
{
  ref: "unique_reference",
  success: true,
  requestKey: "tx_request_key", // Kadena transaction request key
  chainId: "2", // Chain ID the transaction was submitted on
  responseCode: "OK_SUCCESS",
  timestamp: number, // Unix timestamp in milliseconds
}

// KDA Payment Failure
```

```
{
  ref: "unique_reference",
  success: false,
  message: "Insufficient funds", // Error description
  responseCode: "ERR_KDA_FAILED", // or other error codes
}
```

KDA-Specific Error Codes:

ERR_MULTIPLE_RECIPIENTS - KDA payments support single recipient only
ERR_MISSING_PARAMS - Missing recipient account or amount
ERR_KDA_PREP_FAILED - Failed to prepare KDA payment
ERR_KDA_FAILED - KDA transaction failed
Content Creation & Swaps
type: "create-post" (App → Platform)
Submits content to be published by the app. Ensure all media files are correctly referenced. Do not confuse this payload with the response payload. This payload key is simply a key required for this specific command type "create-post"

```
// Message detail object
{
  type: "create-post",
  params: {
    headline: "text",
    nftDescription: "text",
    previewAsset: {type:"image", file:File, preview:"dataURL"},
    extraContents: [{type:"image", file:File, preview:"dataURL"},
{type:"long_text", contentData:"text"}],
    appQuery: { campaign: "cname", tstamp: Date.now(), ref: "internal
reference" },
    appEmbed: { url: "iframe_url", type: "video|audio|shop|game", shape:
"square|landscape" }
  }
}
```

Parameter Details
appEmbed:
Embeds an interactive iframe in the post. Perfect for video players, audio players, mini-games, or shop widgets.

url - The iframe source URL (required)
type - Display style: "video" (play icon), "audio" (music icon), "shop" (cart icon), "game" (gamepad icon). Defaults to "video"
shape - Aspect ratio: "square" (1:1), "landscape" (9:6). Defaults to "square"
previewAsset:

A thumbnail/preview image for your post. Must be provided as an object with type, file, and preview properties.

Example: Creating previewAsset from URL
To use an image from a URL as your preview, you need to fetch it, convert to a File object, and create a dataURL:

```
// Fetch image from URL
const response = await fetch(thumbnailUrl);
const blob = await response.blob();

// Convert Blob to dataURL
const dataURL = await new Promise((resolve, reject) => {
  const reader = new FileReader();
  reader.onloadend = () => resolve(reader.result);
  reader.onerror = reject;
  reader.readAsDataURL(blob);
});

// Convert dataURL to File
const dataURLtoFile = (dataurl, filename) => {
  const arr = dataurl.split(',');
  const mimeMatch = arr[0].match(/:(.*?);/);
  if (!mimeMatch) throw new Error("Invalid data URL");
  const mime = mimeMatch[1];
  const bstr = atob(arr[1]);
  let n = bstr.length;
  const u8arr = new Uint8Array(n);
  while (n--) {
    u8arr[n] = bstr.charCodeAt(n);
  }
  return new File([u8arr], filename, { type: mime });
};

const imageFile = dataURLtoFile(dataURL, 'thumbnail.jpg');

// Create previewAsset object
const previewAsset = {
  type: 'image',
  file: imageFile,
  preview: dataURL
};

// Use in create-post
```

```
sdk._sendCommand({
  type: 'create-post',
  params: {
    headline: 'Check out this video!',
    nftDescription: 'Amazing content',
    previewAsset: previewAsset,
    appEmbed: {
      url: 'https://example.com/embed/video123',
      type: 'video',
      shape: 'square'
    }
  }
});
```

type: "create-post-response" (Platform → App)
Indicates the result of a content submission request. The user can Mint on Blockchain any content any time after it is created.

```
// Response payload object (Success)
{
  success: true,
  responseCode: "OK_SUCCESS",
  postId: "post_id", // Accessible instantly at /post/:post_id
}

// Response payload object (Failure)
{
    success: false,
    error: "Error details",
    responseCode: "ERR_POST_EXCEPTION"
}
```

type: "authorise-swap" (App → Platform)
Creates a swap offer for an NFT you own. This generates a partially signed transaction that allows another user to purchase your NFT for a specified amount. Used in NFT marketplace scenarios with STAS-789 tokens.

```
{
  type: "authorise-swap",
  contractTx: string,    // NFT contract transaction ID
  value: number          // Amount in satoshis you want for the NFT
}
```

How NFT Swaps Work
The STAS-789 swap protocol enables atomic NFT trades:

Seller (authorize-swap): Creates a partially signed offer transaction
Buyer (swap-buy): Completes the transaction by adding payment inputs
Atomic: Either both parties get what they want, or the transaction fails
Platform Role: Validates NFT ownership, creates swap tx, handles confirmations
Requirements: You must own exactly one unspent UTXO of the NFT. The platform verifies ownership before creating the swap offer. Wallet authentication required.
type: "authorise-swap-response" (Platform → App)
Response after creating a swap offer. Contains the partially signed swap transaction hex.

```
// Success response
{
  timestamp: string,
  success: true,
  contractTx: string,        // NFT contract transaction ID
  parsedNFT: {              // Parsed NFT metadata from STAS-789
    protocolId: "STAS-789",
    symbol: string,          // NFT symbol
    tokenId: string,         // Unique token identifier
    // ... other NFT properties
  },
  wanted: {
    satoshis: number         // Amount requested for NFT
  },
  swapHex: string,          // Partially signed swap transaction (hex)
  responseCode: "OK_SUCCESS"
}

// Failure responses
{
  timestamp: string,
  success: false,
  error: string,
  responseCode:
    "ERR_NOT_FOUND" |      // Contract transaction not found
    "ERR_NO_MARKER" |      // Invalid STAS-789 format
    "ERR_PARSE_FAIL" |     // Failed to parse NFT metadata
    "ERR_NO_UTXO" |        // NFT not found in wallet or already spent
    "ERR_USER_CANCEL"      // User rejected the swap offer
}
```

Next Step: Share the swapHex with potential buyers. They use this hex with swap-buy to complete the purchase.

type: "swap-buy" (App → Platform)
Completes an NFT purchase using a swap offer from the seller. Requires the swap hex from the seller's authorize-swap response.

```
{
  type: "swap-buy",
  swapHex: string,       // Partially signed swap tx from seller
  contractTx: string,    // NFT contract transaction ID
  value: number          // Amount in satoshis (must match seller's asking price)
}
```

Swap Buy Process
Platform validates the swap offer and NFT ownership
Fetches required UTXOs from your wallet for payment + fees
Consolidates UTXOs if needed (may require additional transaction)
Completes the partially signed swap transaction with your payment inputs
Verifies NFT transfer and broadcasts the final transaction
NFT transfers to your wallet, satoshis transfer to seller
Requirements: Sufficient BSV balance to cover the NFT price plus transaction fees. The platform automatically handles UTXO consolidation and fee calculation. Wallet authentication required.
Security: Platform validates that the seller still owns the NFT at the moment of purchase. If the NFT was already sold or spent, the transaction will fail with an error.
type: "swap-buy-response" (Platform → App)
Response after attempting to purchase an NFT through a swap.

```
// Success response
{
  timestamp: string,
  success: true,
  contractTx: string,        // NFT contract transaction ID
  swapTxHex: string,        // Final completed swap transaction hex
  responseCode: "OK_SUCCESS"
}

// Failure responses
{
  timestamp: string,
  success: false,
  error: string,
  responseCode:
    "ERR_SWAP_EXEC" |      // Error executing swap (check error message)
    "ERR_NO_MARKER" |      // Invalid STAS-789 format
    "ERR_PARSE_FAIL" |     // Failed to parse NFT metadata
```

```
    "ERR_USER_CANCEL" |     // User rejected the purchase
    "ERR_UNKNOWN"           // Unexpected error
}
```

Common Error Scenarios:

Insufficient funds: Not enough BSV to cover NFT price + fees
NFT already sold: Seller's NFT UTXO was spent by another buyer
Invalid swap offer: Swap hex is malformed or expired
Seller = Buyer: Cannot buy your own NFT
Consolidation failed: Unable to prepare payment UTXOs
Utilities & Simple Commands
type: "open-link" (App → Platform)
Requests to open a specific URL. Ensure the URL is correctly formatted.

```
// Message detail object
{
  type: "open-link",
  url: "https://..."
}
```

💡 Pro Tip: Auto-Intercept Link Clicks

You can automatically convert all external link clicks in your app to use the open-link command. This gives users a confirmation dialog before leaving your app.

Add this to your app's initialization:

```
document.addEventListener('click', (e) => {
  const link = e.target.closest('a');
  if (!link || !link.href) return;

  const url = link.href;
  const currentOrigin = window.location.origin;
  const linkOrigin = new URL(url).origin;

  // Only intercept external links
  if (linkOrigin !== currentOrigin) {
    e.preventDefault();
    sendCommand({ type: 'open-link', url: url });
  }
}, true);
```

✓ Works for all links in your app
✓ Internal navigation works normally

✓ External links show user confirmation

type: "open-link-response" (Platform → App)
Response to an open-link command, indicating success or failure.

```
{
    appId: string,
    timestamp: number, // Unix timestamp in milliseconds
    success: boolean,
    responseCode: string,
    error?: string // Present if an error occurred
}
```

type: "qr-scan" (App → Platform)
Requests to open the QR code scanner overlay. The platform will display a camera feed for scanning QR codes and return the scanned data.

```
{
  type: "qr-scan",
  ref: "unique_reference" // A unique ID to track this scan request
}
```

type: "qr-scan-response" (Platform → App)
Response when a QR code is successfully scanned. The platform automatically attempts to parse JSON content if detected.

// Successful scan response

```
{
  ref: "unique_reference",
  timestamp: number, // Unix timestamp in milliseconds
  success: true,
  scanData: {
    rawValue: "scanned_text",    // Raw text from QR code
    parsed?: object              // Parsed JSON object (if QR contained
valid JSON)
  },
  responseCode: "OK_SUCCESS"
}

// Failed scan response (no data detected)
{
  ref: "unique_reference",
  timestamp: number, // Unix timestamp in milliseconds
  success: false,
```

```
  scanData: null,
  responseCode: "ERR_NO_DATA"
}
```

type: "qr-scan-stop" (App → Platform)
Requests to close the QR scanner overlay programmatically. Useful for stopping the scanner from within your app logic.

```
{
  type: "qr-scan-stop"
}
```

type: "qr-scan-stop-response" (Platform → App)
Response when the QR scanner is closed either by user action or programmatic request. This indicates the scanning session has ended without a successful scan.

```
{
  ref: "unique_reference",
  timestamp: number, // Unix timestamp in milliseconds
  success: false,
  message: "QR scanning closed by user", // Reason for stopping
  responseCode: "ERR_ABORTED"
}
```

type: "write-clipboard" (App → Platform)
Copies text to the user's clipboard. This command does not require a response and executes immediately. The text is automatically sanitized to remove HTML tags for security.

```
{
  type: "write-clipboard",
  text: string // The text to copy to clipboard
}
```

Note: This command has no response - it executes silently and returns immediately.

QR Code Scanning
type: "qr-scan" (App → Platform)
Requests to open the QR code scanner overlay. The platform displays a camera feed and continuously scans for QR codes. The scanner remains open and continues scanning until explicitly stopped.

```
{
  type: "qr-scan",
  ref: "unique_reference" // A unique ID to track this scan request
}
```

type: "qr-scan-response" (Platform → App)
Sent for EACH QR code scan attempt. The scanner overlay remains open after sending this response, allowing continuous scanning. The platform automatically sanitizes content and attempts to parse JSON if detected.

```
// Successful scan response
{
  ref: "unique_reference",
  timestamp: number, // Unix timestamp in milliseconds
  success: true,
  scanData: {
    rawValue: "scanned_text",    // Sanitized raw text from QR code
    parsed?: object              // Parsed JSON (if valid JSON detected)
  },
  responseCode: "OK_SUCCESS"
}

// Failed scan response (no readable data detected)
{
  ref: "unique_reference",
  timestamp: number, // Unix timestamp in milliseconds
  success: false,
  scanData: null,
  responseCode: "ERR_NO_DATA"
}
```

Important: After receiving a successful scan, the scanner stays open. Your app should call qr-scan-stop to close it.

type: "qr-scan-stop" (App → Platform)
Closes the QR scanner overlay programmatically. Use this after receiving a successful scan or when you want to stop scanning from your app logic. This command does not send a response.

```
{
  type: "qr-scan-stop"
}
```

Note: This command does not trigger a response. The scanner simply closes.

type: "qr-scan-stop-response" (Platform → App)
Sent ONLY when the user manually closes the scanner overlay (by clicking close/back button). This indicates the scanning session ended by user action, not by a successful scan or programmatic stop.

```
{
  ref: "unique_reference",
```

```
  timestamp: number, // Unix timestamp in milliseconds
  success: false,
  message: "QR scanning closed by user", // Reason for stopping
  responseCode: "ERR_ABORTED"
}
```

Usage pattern: Listen for this response to handle the case where the user cancels scanning. If you programmatically call qr-scan-stop, you will NOT receive this response.

Transactions & History
type: "full-transaction" (App → Platform)
Requests complete transaction data including raw hex and BUMP proof for SPV verification. This is useful for apps that need to verify transactions independently or submit them to other services.

```
{
  type: "full-transaction",
  txid: string,          // Transaction ID to retrieve
  ref: "unique_ref"      // Unique reference to track this request
}
```

What is BUMP?
BUMP (Bitcoin Unified Merkle Path) is a standardized format for Merkle proofs that enables lightweight SPV (Simplified Payment Verification) without requiring a full blockchain node.

Use Cases: Verifying transactions, submitting to overlay services, building SPV wallets, cross-chain bridges, or any scenario requiring cryptographic proof of transaction inclusion in a block.

🎉 Beta Period: Full transaction retrieval with BUMP proofs is currently free during the beta period. Take advantage of this to build and test your SPV applications!
Authentication Required: This endpoint requires wallet authentication. Unauthenticated requests will prompt the user to connect their wallet.
type: "full-transaction-response" (Platform → App)
Response containing complete transaction data with BUMP proof.

```
// Success response
{
  ref: "unique_ref",
  timestamp: number,
  success: true,
  txid: string,              // Transaction ID
  tx_hex: string,            // Raw transaction hex (can be null if not
found)
  bump_hex: string,          // BUMP proof hex (can be null if not
confirmed)
  responseCode: "OK_SUCCESS"
```

```
}

// Failure response
{
  ref: "unique_ref",
  timestamp: number,
  success: false,
  error: string,
  responseCode: "ERR_TX_NOT_FOUND" // or "ERR_MISSING_TXID"
}
```

Note: tx_hex and bump_hex may be null if the transaction hasn't been confirmed yet or if it's not in our overlay index.

type: "token-history" (App → Platform)
Requests transaction history for a specific token. Supports both BSV and ICP protocols with consistent pagination parameters. This command is designed for apps running within the iframe to retrieve token transaction data.

```
// BSV Token History Request (Default - no token field)
{
  type: "token-history",
  // No token field = defaults to BSV protocol
  offset: 0,    // Starting position for pagination (0-based)
  limit: 100    // Number of transactions to retrieve (max 100)
}

// ICP Token History Request
{
  type: "token-history",
  token: {
    protocol: "ICP",
    specification: {
      indexCanisterId: "qhbym-qaaaa-aaaaa-aaafq-cai"  // ICP index canister
ID
    }
  },
  offset: 0,    // Starting position for pagination (0-based)
  limit: 50     // Number of transactions to retrieve (max 100)
}
```

Pricing Notice: This endpoint is free during the beta period. A small per-request fee or inclusion in a subscription tier will apply in a future release. Plan your integration accordingly.
type: "token-history-response" (Platform → App)

Response containing the requested token transaction history. The format varies by protocol but maintains consistent structure for success/error handling.

```
// Successful BSV History Response
{
  ref: "unique_reference", // Echoed from request if provided
  success: true,
  protocol: "BSV",
  transactions: [
    {
      id: "txid_hash", // Transaction ID
      amount: 50000, // Amount in satoshis
      ts: 1693737600, // Timestamp in seconds (blockHeight or timestamp)
      note: "Payment for services", // OP_RETURN data or memo
      type: "received" // Transaction type: "sent", "received", etc.
      // Note: BSV uses UTXOs, not account-based addresses
      // Use full-transaction to analyze inputs/outputs if needed
    }
  ],
  pagination: {
    offset: 0,
    limit: 100,
    total: 156,
    hasMore: true
  },
  responseCode: "OK_SUCCESS"
}

// Successful ICP History Response
{
  ref: "unique_reference", // Echoed from request if provided
  success: true,
  protocol: "ICP",
  indexCanisterId: "qhbym-qaaaa-aaaaa-aaafq-cai",
  transactions: [
    {
      id: 12345, // Transaction index
      amount: 100000000, // Amount in token's smallest unit
      ts: 1693737600, // Timestamp in seconds
      note: "Payment memo", // Decoded memo from transfer
      type: "received", // Transaction type sent/received
      from: "principal_id_sender", // Sender principal ID
      to: "principal_id_recipient" // Recipient principal ID
```

```
    }
  ],
  pagination: {
    offset: 0,
    limit: 50,
    total: 42,
    hasMore: true
  },
  responseCode: "OK_SUCCESS"
}

// Failed History Response
{
  ref: "unique_reference", // Echoed from request if provided
  success: false,
  protocol: "BSV" | "ICP",
  error: "Token not found or access denied",
  responseCode: "ERR_TOKEN_NOT_FOUND" // or other error codes
}
```

Protocol-Specific Notes:

Standardized Format: All transactions from both BSV and ICP are returned in a unified format with consistent field names
ID Field: Contains txid (hash) for BSV transactions, transaction index (number) for ICP transactions
Amount Field: Satoshis for BSV, token's smallest unit for ICP (e.g., e8s for ICP tokens)
Timestamp (ts): Relative timestamp - blockHeight for BSV, seconds for ICP
Note Field: OP_RETURN data for BSV, decoded memo string for ICP transfers
Type Field: Transaction direction/type (sent, received)
BSV Transactions: BSV is UTXO-based (not account-based), so from/to addresses are not included. Use full-transaction to analyze inputs and outputs if needed
ICP Transactions: From/To fields contain principal IDs for account-based ICP transfers
Pagination: Consistent offset/limit parameters across both protocols for easy developer integration
Geolocation
type: "geolocation" (App → Platform)
Requests the user's geolocation with high accuracy tracking. The platform will continuously monitor location accuracy and send updates as precision improves, with a target accuracy of 100 meters.

```
{
  type: "geolocation",
  ref: "unique_ref"      // Unique reference to track this request
}
```

How Geolocation Works
Progressive Accuracy: The platform starts with available location data and continuously improves

precision
Multiple Updates: Your app will receive multiple responses as accuracy improves (isFinal: false)
Final Response: When accuracy reaches ≤100m or after timeout, a final response is sent (isFinal: true)
Heartbeat: Updates sent every 30 seconds even if accuracy hasn't improved
Automatic Cleanup: Location tracking stops automatically after final response or if app sends a new geolocation request
Permission Required: This requires browser geolocation permission. The user will be prompted if permission hasn't been granted yet.
type: "geolocation-response" (Platform → App)
Response containing geolocation data. Your app may receive multiple responses for a single request as accuracy improves.

```
// Progressive update (accuracy improving)
{
  ref: "unique_ref",
  timestamp: number,
  location: {
    latitude: number,      // Latitude in degrees
    longitude: number,     // Longitude in degrees
    accuracy: number       // Accuracy in meters (e.g., 450.5)
  },
  isFinal: false,         // More updates coming as accuracy improves
  responseCode: "OK_SUCCESS"
}

// Final response (target accuracy reached)
{
  ref: "unique_ref",
  timestamp: number,
  location: {
    latitude: number,
    longitude: number,
    accuracy: number       // ≤100 meters
  },
  isFinal: true,          // No more updates for this request
  responseCode: "OK_SUCCESS"
}

// Error response
{
  ref: "unique_ref",
  timestamp: number,
  error: string,
```

```
  isFinal: true,
  responseCode: "ERR_NOT_SUPPORTED" // or "ERR_PERMISSION_DENIED"
}
```

Best Practice: Listen for multiple responses and update your UI as accuracy improves. Check the isFinal flag to know when location tracking has stopped.

Example Usage

Example 1: Initializing Connection (JS SDK)

Using the JS SDK, you can connect and listen for the response easily.

```
// Assumes platformSDK is available on the window object

// 1. Send the connection request
platformSDK.sendCommand({
  type: "connection",
  navbg: "#1a202c" // Suggest a dark nav bar
});

// 2. Listen for the response from the platform
platformSDK.onCommand((data) => {
  if (data.type === "connection-response") {
    if (data.payload && data.payload.wallet) {
      console.log("Platform connected!", data.payload);
      // Store wallet details, etc.
    } else {
      console.error("Connection failed or returned invalid data.");
    }
  }
});
```

Example 2: Making a Payment (Vanilla JS)

This example shows how to request a payment and handle the response using the Core Protocol directly, without the JS SDK.

```
// 1. Define the payment details
const paymentRef = 'my-app-tx-' + Date.now();
const paymentDetails = {
  type: "pay",
  ref: paymentRef,
  recipients: [
    { reason: "AI_IMG" }, // Pay platform fee for image generation
    { address: "1...", value: 50000, note: "Creator tip" }
  ]
};
```

```
// 2. Add a listener for the specific response
const handlePaymentResponse = (event) => {
  const { command, type, payload } = event.data;
  if (command === "ninja-app-command" && type === "pay-response" &&
payload.ref === paymentRef) {
    if (payload.success) {
      console.log("Payment successful! TxHex:", payload.rawTxHex);
      // Now you can proceed with the action the payment was for
    } else {
      console.error("Payment failed:", payload.message);
    }
    // Clean up the listener after handling the response
    window.removeEventListener('message', handlePaymentResponse);
  }
};
window.addEventListener('message', handlePaymentResponse);

// 3. Send the message to the platform
window.parent.postMessage({ command: "ninja-app-command", detail:
paymentDetails }, "*");
```

Example 3: QR Code Scanning (JS SDK)

This example demonstrates how to initiate QR scanning and handle different response scenarios.

```
// Assumes platformSDK is available on the window object

// 1. Start QR scanning
const scanRef = 'qr-scan-' + Date.now();
platformSDK.sendCommand({
  type: "qr-scan",
  ref: scanRef
});

// 2. Listen for scan results
platformSDK.onCommand((data) => {
  if (data.type === "qr-scan-response" && data.payload.ref === scanRef) {
    if (data.payload.success) {
      console.log("QR Code scanned successfully!");
      console.log("Raw text:", data.payload.scanData.rawValue);

      // Check if the QR code contained JSON data
      if (data.payload.scanData.parsed) {
        console.log("Parsed JSON:", data.payload.scanData.parsed);
```

```
        // Handle structured data from QR code
        handleStructuredData(data.payload.scanData.parsed);
      } else {
        // Handle plain text from QR code
        handlePlainText(data.payload.scanData.rawValue);
      }
    } else {
      console.log("No QR code data detected");
    }
  } else if (data.type === "qr-scan-stop-response" && data.payload.ref ===
scanRef) {
    console.log("QR scanning was cancelled:", data.payload.message);
    // Handle cancellation - maybe show user message or reset UI
  }
});

// 3. Optional: Stop scanning programmatically
function stopScanning() {
  platformSDK.sendCommand({
    type: "qr-scan-stop"
  });
}

// Example handlers for different QR code content types
function handleStructuredData(jsonData) {
  // Example: Handle payment request QR codes
  if (jsonData.type === "payment-request") {
    initiatePayment(jsonData.address, jsonData.amount);
  }
  // Example: Handle app-specific data
  else if (jsonData.appData) {
    processAppData(jsonData.appData);
  }
}

function handlePlainText(text) {
  // Example: Handle Bitcoin addresses
  if (text.match(/^[13][a-km-zA-HJ-NP-Z1-9]{25,34}$/)) {
    console.log("Bitcoin address detected:", text);
  }
  // Example: Handle URLs
  else if (text.startsWith('http')) {
    console.log("URL detected:", text);
```

```
    // Maybe open link or preview content
  }
  // Example: Handle general text
  else {
    console.log("General text content:", text);
  }
}
```

Example 4: Geolocation Tracking (JS SDK)
Track user location with progressive accuracy updates.

```
// Assumes platformSDK is available on the window object

let locationRef = 'location-' + Date.now();
let bestAccuracy = Infinity;

// 1. Request geolocation
platformSDK.sendCommand({
  type: "geolocation",
  ref: locationRef
});

// 2. Listen for location updates
platformSDK.onCommand((data) => {
  if (data.type === "geolocation-response" && data.payload.ref ===
locationRef) {
    if (data.payload.location) {
      const { latitude, longitude, accuracy } = data.payload.location;

      console.log(`Location update: (${latitude}, ${longitude})`);
      console.log(`Accuracy: ${accuracy}m (best: ${bestAccuracy}m)`);

      // Track best accuracy
      if (accuracy < bestAccuracy) {
        bestAccuracy = accuracy;
        updateMap(latitude, longitude, accuracy);
      }

      // Check if this is the final update
      if (data.payload.isFinal) {
        console.log("Final location received:", {
          lat: latitude,
          lng: longitude,
          accuracy: accuracy
```

```
        });
        // Use the location data
        submitLocationToBackend(latitude, longitude);
      } else {
        console.log("Location improving... waiting for better accuracy");
      }
    } else if (data.payload.error) {
      console.error("Geolocation error:", data.payload.error);
      // Handle permission denied or not supported
    }
  }
});

// Helper: Update map display
function updateMap(lat, lng, accuracy) {
  // Update your map UI with new position
  // Show accuracy radius circle
  console.log(`Updating map to (${lat}, ${lng}) ±${accuracy}m`);
}

// Helper: Send to backend
function submitLocationToBackend(lat, lng) {
  fetch('/api/location', {
    method: 'POST',
    body: JSON.stringify({ latitude: lat, longitude: lng }),
    headers: { 'Content-Type': 'application/json' }
  });
}
```

Example 5: Full Transaction with BUMP (JS SDK)

Retrieve complete transaction data for SPV verification or re-broadcasting.

```
// Assumes platformSDK is available on the window object

const txid = "abc123..."; // Transaction ID to retrieve
const txRef = 'tx-' + Date.now();

// 1. Request full transaction
platformSDK.sendCommand({
  type: "full-transaction",
  txid: txid,
  ref: txRef
});
```

```
// 2. Listen for response
platformSDK.onCommand((data) => {
  if (data.type === "full-transaction-response" && data.payload.ref ===
txRef) {
    if (data.payload.success) {
      const { tx_hex, bump_hex } = data.payload;

      if (tx_hex && bump_hex) {
        console.log("Complete transaction retrieved");
        console.log("Raw TX:", tx_hex.substring(0, 64) + "...");
        console.log("BUMP proof:", bump_hex.substring(0, 64) + "...");

        // Use cases:
        // 1. Verify transaction with BUMP proof
        verifyTransactionWithBUMP(tx_hex, bump_hex);

        // 2. Submit to another overlay service
        submitToOverlay(tx_hex, bump_hex);

        // 3. Store for offline verification
        storeForOfflineVerification(txid, tx_hex, bump_hex);
      } else {
        console.log("Transaction found but not confirmed yet");
        // tx_hex or bump_hex might be null if unconfirmed
      }
    } else {
      console.error("Transaction not found:", data.payload.error);
    }
  }
});

// Helper: Verify with BUMP
function verifyTransactionWithBUMP(txHex, bumpHex) {
  // Use BUMP library to verify Merkle proof
  // This proves the transaction is included in a block
  console.log("Verifying transaction with BUMP proof...");
  // Implementation depends on your BUMP library
}

// Helper: Submit to overlay
function submitToOverlay(txHex, bumpHex) {
  // Submit to external overlay service
  fetch('https://overlay-service.example/submit', {
```

```
    method: 'POST',
    body: JSON.stringify({ rawTx: txHex, bump: bumpHex }),
    headers: { 'Content-Type': 'application/json' }
  });
}
```

Example 6: NFT Marketplace - Create Swap Offer (JS SDK)
Create a swap offer to sell an NFT you own.

```
// Assumes platformSDK is available on the window object

const nftContractTx = "abc123..."; // Your NFT contract transaction ID
const askingPrice = 100000; // 100,000 satoshis (0.001 BSV)
const swapRef = 'swap-auth-' + Date.now();

// 1. Request to create swap offer
platformSDK.sendCommand({
  type: "authorise-swap",
  contractTx: nftContractTx,
  value: askingPrice
});

// 2. Listen for response
platformSDK.onCommand((data) => {
  if (data.type === "authorise-swap-response") {
    if (data.payload.success) {
      const swapOffer = {
        swapHex: data.payload.swapHex,
        contractTx: data.payload.contractTx,
        nft: data.payload.parsedNFT,
        price: data.payload.wanted.satoshis
      };

      console.log("Swap offer created!");
      console.log("NFT:", swapOffer.nft.symbol, swapOffer.nft.tokenId);
      console.log("Price:", swapOffer.price, "satoshis");

      // Share swap offer
      // Option 1: Store in your database
      saveSwapOfferToDatabase(swapOffer);

      // Option 2: Display listing on marketplace
      displayListing(swapOffer);
```

```
      // Option 3: Share swap hex with specific buyer
      shareWithBuyer(swapOffer.swapHex, swapOffer.contractTx,
swapOffer.price);

    } else {
      console.error("Swap creation failed:", data.payload.error);

      // Handle specific errors
      if (data.payload.responseCode === "ERR_NO_UTXO") {
        alert("You don't own this NFT or it's already listed");
      } else if (data.payload.responseCode === "ERR_USER_CANCEL") {
        console.log("User cancelled swap creation");
      }
    }
  }
});

// Helper: Save to database
function saveSwapOfferToDatabase(offer) {
  fetch('/api/listings', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      contractTx: offer.contractTx,
      swapHex: offer.swapHex,
      nftSymbol: offer.nft.symbol,
      tokenId: offer.nft.tokenId,
      price: offer.price,
      seller: offer.nft.issuer
    })
  });
}

// Helper: Display in UI
function displayListing(offer) {
  const listingHTML = `
    <div class="nft-listing">
      <h3>${offer.nft.symbol}</h3>
      <p>Token ID: ${offer.nft.tokenId}</p>
      <p>Price: ${offer.price} sats</p>
      <button onclick="copySwapOffer('${offer.swapHex}',
'${offer.contractTx}', ${offer.price})">
        Copy Swap Data
```

```
      </button>
    </div>
  `;
  document.getElementById('listings').innerHTML += listingHTML;
}
```

Example 7: NFT Marketplace - Buy with Swap (JS SDK)
Purchase an NFT using a swap offer from the seller.

```
// Assumes platformSDK is available on the window object

// Get swap offer from marketplace listing or shared by seller
const swapOffer = {
  swapHex: "0200000001...", // From seller's authorise-swap response
  contractTx: "abc123...",  // NFT contract transaction
  value: 100000             // Price in satoshis
};

// 1. Request to buy NFT
platformSDK.sendCommand({
  type: "swap-buy",
  swapHex: swapOffer.swapHex,
  contractTx: swapOffer.contractTx,
  value: swapOffer.value
});

// 2. Listen for response
platformSDK.onCommand((data) => {
  if (data.type === "swap-buy-response") {
    if (data.payload.success) {
      console.log("NFT purchased successfully!");
      console.log("Swap transaction:", data.payload.swapTxHex);

      // Update UI
      showSuccessMessage("NFT is now yours!");

      // Refresh NFT collection
      loadMyNFTs();

      // Remove from marketplace
      removeListingFromMarketplace(swapOffer.contractTx);

    } else {
      console.error("Purchase failed:", data.payload.error);
```

```
      // Handle specific errors
      switch(data.payload.responseCode) {
        case "ERR_USER_CANCEL":
          console.log("User cancelled purchase");
          break;
        case "ERR_SWAP_EXEC":
          // Check specific error message
          if (data.payload.error.includes("can not match")) {
            alert("You cannot buy your own NFT");
          } else if (data.payload.error.includes("funds")) {
            alert("Insufficient funds for purchase");
          } else if (data.payload.error.includes("processing")) {
            alert("NFT is being processed, try again in a moment");
          } else {
            alert("Purchase failed: " + data.payload.error);
          }
          break;
        case "ERR_NO_MARKER":
        case "ERR_PARSE_FAIL":
          alert("Invalid NFT format");
          break;
        default:
          alert("Purchase failed: " + data.payload.error);
      }
    }
  }
});

// Helper: Show success
function showSuccessMessage(message) {
  const notification = document.createElement('div');
  notification.className = 'success-notification';
  notification.textContent = message;
  document.body.appendChild(notification);
  setTimeout(() => notification.remove(), 3000);
}

// Helper: Refresh NFT collection
function loadMyNFTs() {
  fetch('/api/my-nfts')
    .then(res => res.json())
    .then(nfts => displayNFTCollection(nfts));
```

```
}

// Helper: Remove from marketplace
function removeListingFromMarketplace(contractTx) {
  fetch(`/api/listings/${contractTx}`, {
    method: 'DELETE'
  });
}
```

Response Codes & Meanings

OK_SUCCESS: Operation completed successfully.

ERR_POST_FAIL: Post creation failed due to incorrect input or errors.

ERR_POST_EXCEPTION: Exception occurred during posting.

ERR_ABORTED: Operation was aborted by the user or process.

ERR_UNKNOWN: An unknown error occurred.

ERR_REJECTED: The request was rejected by the host platform.

ERR_NO_DATA: QR scanner couldn't detect any readable data.

ERR_KDA_PREP_FAILED: Failed to prepare KDA payment (e.g., invalid wallet state).

ERR_KDA_FAILED: KDA transaction execution failed on-chain.

Important Notes

Security: For production apps, always verify the event.origin in your message listeners to ensure messages are coming from Metanet.page.

Tracking Requests: When sending a "pay" message that expects a response, always include a unique ref property. Use this ref to match incoming responses to your original requests.

App Assets: All apps are expected to have a logo.jpeg and a cover.jpeg image in their root directory. These are used for the app's icon and cover image in the platform UI.

Navigation Bar Layout: The platform's top navigation bar covers the top 70px of the screen. Your app design should account for this by providing an appropriate background behind the navigation area and ensuring all app content and interactive elements begin after the top 70px. Use padding-top: 70px on your main body content or margin-top: 70px for overlay elements. The background behind the navigation can be customized to match your app's design (white or any color of your choice).

Protocol is Everything: Remember that the JavaScript SDK is a convenience. The underlying postMessage protocol is the source of truth and can be implemented in any language or framework.